MP3: Page Manager I

Mohammed Sharique
UIN: 837004870
CSCE611: Operating System

02/22/2026

Assigned Tasks

Main: Completed.

System Design

The goal of this machine problem was to implement virtual memory precisely implementing two level paging.

- On initialising the page table I am requesting a frame from kernel memory pool to store the page directory of the page table. I am then requesting another frame from the kernel pool to populate page table pages for the directly mapped memory segments 0MB to 4MB, the entries created have their valid bit (0th bit) set and their indexes are same as the frame number they are pointing to, to achieve direct mapping. And finally I point the first entry of the page directory to this page table.
- While loading the created page table I write the page directory address in the cr3 register.
- To enable paging I set the 32nd bit (leftmost bit) of the cr0 register.
- I am handling page faults by first getting the page fault address from the cr2 register (32 its). Then I extract the page directory index (leftmost 10 bits), page table index (next 10 bits) and offset (remaining 12 bits). First I index the page directory and check if the page directory entry is valid or not. If the entry is not valid I create a new page table, populate it with invalid entries (setting the last bit to 0). If not, I do nothing. Then I get the page table page entry using the page table index. If the page is invalid I request a frame from process memory pool and point this entry to the received frame. If the page is valid, I throw an error and panic.

Code Description

I have added content to only page_table.C. I ran linter on kernel.C and page_table.H .
Github: https://github.com/sharique-tamu/csce-611-machine-problems
The code should compile by just running make in the folder containing the code.

page_table.C:init_paging : This function initialises kernel_mem_pool, process_mem_pool and shared_size variables for the page table. It also asserts that the shared size is 4MB as some part of code is written assuming that shared size is 4MB.

```
void PageTable::init_paging(ContFramePool *_kernel_mem_pool,
                            ContFramePool *_process_mem_pool,
                            const unsigned long _shared_size) {
  kernel_mem_pool = _kernel_mem_pool;
  process_mem_pool = _process_mem_pool;
  shared_size = _shared_size;
  assert(shared_size == 4 MB);
  Console::puts("Initialized Paging System\n");
}
```

page_table.C:PageTable() (constructor) : This constructor initializes a new page table by allocating one frame for the page directory and one frame for the first page table from the kernel memory pool. The page table is filled to map the shared region (4 MB), mapping each virtual address i * 4 KB to the same physical address with the Valid and Read/Write bits set. The first page directory entry is updated to point to this page table, enabling the direct mapping. All remaining directory entries are marked as invalid so that accesses outside the shared region will trigger page faults.

```
PageTable::PageTable() {
  // Initialize page directory
  unsigned long frame = kernel_mem_pool->get_frames(1);
  page_directory = (unsigned long *)(frame * 4 KB);

  // Initialize first page table for direct mapping of shared size
  frame = kernel_mem_pool->get_frames(1);
  unsigned long address = 0;

  unsigned long *page_table = (unsigned long *)(frame * 4 KB);

  for (int i = 0; i < shared_size / (4 KB); i++) {
    // Enabling R/W bit and Valid bit
    page_table[i] = address | 3;
    address += 4 KB;
  }

  page_directory[0] = (unsigned long)page_table | 3;
  for (int i = 1; i < 1024; i++) {
    // Marking them as read write and invalid
    page_directory[i] = 0 | 2;
  }

  Console::puts("Constructed Page Table object\n");
}
```

page_table.C:load : This function activates the page table by making it the current address translation structure. It sets current_page_table to the current object and loads the physical address of its page directory into the CR3 register. Writing to CR3 tells the CPU to use this page directory for virtual-to-physical address translation.

page_table.C:enable_paging : This function enables hardware paging on the CPU. It sets the highest(32nd) bit of the CR0 register. Setting this bit turns on paging, causing the processor to translate virtual addresses using the page tables referenced by CR3. It then sets paging_enabled variable, that is used for local references.

```
void PageTable::load() {
    current_page_table = this;
    write_cr3((unsigned long)page_directory);
    Console::puts("Loaded page table\n");
}
```

```
void PageTable::enable_paging() {
    write_cr0(read_cr0() | 0x80000000);
    paging_enabled = 1;
    Console::puts("Enabled paging.\n");
}
```

page_table.C:handle_fault : This function handles a page fault by allocating the required paging structures on demand. It reads the faulting virtual address from CR2 and extracts the page directory and page table indices. It then checks whether the corresponding page directory entry is present. If not, it allocates a new frame for a page table, initializes all its entries as invalid, and updates the page directory to point to it. Then it checks the specific page table entry for the faulting address. If the entry is already marked present, it treats this as an unexpected fault and triggers an assertion. Otherwise, it allocates a new frame from the process memory pool and updates the page table entry to map the faulting virtual page to that physical frame with the Valid and Read/Write bits set.

```
void PageTable::handle_fault(REGS *_r) {
  unsigned long fault_address = read_cr2();

  unsigned long dir_idx = fault_address >> 22;
  unsigned long pt_idx = (fault_address << 10) >> 22;
  unsigned long offset = fault_address & ((1 << 12) - 1);

  unsigned long pde_entry = current_page_table->page_directory[dir_idx];

  unsigned long *page_table;
  unsigned long pte_entry;
  if ((pde_entry & 1) == 0) {
    unsigned long new_frame = kernel_mem_pool->get_frames(1);
    page_table = (unsigned long *)(new_frame * 4 KB);

    for (int i = 0; i < 1024; i++) {
      // Enabling R/W bit and invalid bit
      page_table[i] = 0 | 2;
    }
    current_page_table->page_directory[dir_idx] = (new_frame << 12) | 3;
    pte_entry = 0 | 2;
  } else {
    unsigned long pt_frame = pde_entry >> 12;
    page_table = (unsigned long *)(pt_frame * 4 KB);
    pte_entry = page_table[pt_idx];
  }

  if (pte_entry & 1) {
    Console::puts("A valid page table from a valid page directroy throws "
                  "handle fault error\n");
    assert(false);
  } else {
    unsigned long new_frame = process_mem_pool->get_frames(1);
    page_table[pt_idx] = (new_frame << 12) | 3;
  }
}
```

Testing

For this MP I completely relied on the provided tests. I played around with writing data on memory segment 0MB to 4MB and it did break the program and the running kernel, confirming that memory address were directly mapped. After enabling paging the program instructions ran sequentially also

confirming that address is mapped directly. On running the provided test the no. of page faults generated are equal to no. of memory references made and the memory becomes available after the page fault is raised confirming paging is enabled and page faults are handled correctly.